

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: "Информационная безопасность / Information security"

Дисциплина:

«Информационная безопасность баз данных»

КУРСОВАЯ РАБОТА

**на тему «Проектирование и реализация защищенной базы данных
для информационной системы «Проката электросамокатов»**

Выполнил студент:

группа/поток N3346

Суханкулиев Мухаммет / _____

ФИО

Подпись

Проверил:

Салихов Максим Русланович / _____

ФИО

Подпись

*Отметка о выполнении (один из вариантов:
отлично, хорошо, удовлетворительно)*

Дата

Санкт-Петербург

2025 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

Студент	Суханкулиев Мухаммет
	(Фамилия, И. О.)
Факультет	Безопасности информационных технологий
Группа	N3346
Направление (специальность)	10.03.01 Информационная безопасность
Руководитель	Салихов Максим Русланович, преподаватель
	(Фамилия, И.О., должность, ученое звание, степень)
Дисциплина	Информационная безопасность баз данных
Наименование темы	Проектирование и реализация защищенной базы данных для информационной системы «Проката электросамокатов»

Задание	Разработать защищенную базу данных и API-сервис для ИС «Прокат электросамокатов» с механизмами аутентификации.
---------	--

Краткие методические указания	Выполнить проектирование, реализацию и защиту БД в PostgreSQL (мониторинг, RBAC, шифрование). Разработать API-сервис с использованием ORM и настроить систему резервного копирования.
-------------------------------	---

Содержание пояснительной записки	Инфологическое моделирование; реализация БД в PostgreSQL; защита данных (мониторинг, RBAC, шифрование); реализация API-сервиса; резервное копирование и восстановление; анализ реализованных мер защиты.
----------------------------------	--

Рекомендуемая литература	Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с.
--------------------------	--

Руководитель	Подпись, дата
Студент	Подпись, дата

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ГРАФИК ВЫПОЛНЕНИЯ КУРСОВОГО ПРОЕКТА (РАБОТЫ)

Студент	Суханкулиев Мухаммет
	(Фамилия, И. О.)
Факультет	Безопасности информационных технологий
Группа	N3346
Направление (специальность)	10.03.01 Информационная безопасность
Руководитель	Салихов Максим Русланович, преподаватель
	(Фамилия, И.О., должность, ученое звание, степень)
Дисциплина	Информационная безопасность баз данных
Наименование темы	Проектирование и реализация защищенной базы данных для информационной системы «Проката электросамокатов»

№ п/п	Наименование этапа	Дата завершения		Оценка и подпись руководителя
		Планируемая	Фактическая	
1	Инфологическое моделирование баз данных по методу «сущность-связь»	01.11.2025	27.09.2025	
2	Реализация БД в рамках СУБД	04.11.2025	04.10.2025	
3	Защита баз данных	04.11.2025	04.10.2025	
4	Реализация сервиса для взаимодействия с разработанной базой данных	13.12.2025	11.10.2025	
5	Резервирование БД и восстановление по контрольным точкам	13.12.2025	11.10.2025	

Руководитель

Подпись, дата

Студент

Подпись, дата

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

АННОТАЦИЯ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

Студент	Суханкулиев Мухаммет
	(Фамилия, И. О.)
Факультет	Безопасности информационных технологий
Группа	N3346
Направление (специальность)	10.03.01 Информационная безопасность
Руководитель	Салихов Максим Русланович, преподаватель
	(Фамилия, И.О., должность, ученое звание, степень)
Дисциплина	Информационная безопасность баз данных
Наименование темы	Проектирование и реализация защищенной базы данных для информационной системы «Проката электросамокатов»

ХАРАКТЕРИСТИКА КУРСОВОГО ПРОЕКТА (РАБОТЫ)

1. Цель и задачи работы

Предложены студентом	Сформулированы при участии студента	Определены руководителем
☒	☐	☐

2. Характер работы

Расчет	Моделирование	Конструирование	Другое
☐	☒	☒	☐

3. Содержание работы

Выполнен полный цикл разработки: спроектирована и реализована в PostgreSQL база данных для ИС «Прокат электросамокатов». Внедрены механизмы защиты (аудит, шифрование, RBAC) и отказоустойчивости (резервное копирование). Разработан API-сервис на Python/Flask с аутентификацией на основе JWT.

4. Выводы

Разработана защищенная база данных и API-сервис для ИС «Прокат электросамокатов». Практически подтверждена эффективность реализованных механизмов безопасности и процедур обеспечения отказоустойчивости.

Руководитель

Подпись, дата

Студент

Подпись, дата

«__» _____ 2025 г.

СОДЕРЖАНИЕ

Список сокращений и условных обозначений	7
Термины и определения.....	8
Введение	9
1 Системный анализ информационной системы «Прокат электросамокатов»	11
1.1 Описание процессов и задач.....	11
1.2 Выделение сущностей и построение ER-диаграммы.....	12
1.3 Приведение схемы отношений к третьей нормальной форме	15
1.4 Моделирование уровня представлений.....	16
2 Реализация базы данных в рамках СУБД PostgreSQL	18
2.1 Обоснование выбора СУБД.....	18
2.2 Создание и наполнение таблиц	18
2.3 Реализация индексов и внешних ключей	23
2.4 Создание представлений.....	24
2.5 Тестирование базы данных.....	25
3 Обеспечение безопасности базы данных	28
3.1 Мониторинг операций в БД на основе триггеров	28
3.2 Шифрование конфиденциальных данных	31
3.3 Разграничение доступа на основе ролевой модели (RBAC)	33
4 Реализация сервиса для взаимодействия с разработанной базой данных	35
4.1 Обоснование выбора стека технологий.....	35
4.2 Архитектура сервиса и описание API.....	36
5 Резервирование базы данных и восстановление по контрольным точкам	39
5.1 Создание резервной копии (контрольной точки).....	39
5.2 Демонстрация процесса восстановления	39
6 Аудит безопасности реализованного решения	44
6.1 Анализ реализованных методов защиты.....	44
6.2 Преимущества и недостатки выбранных решений	45
6.3 Рекомендации по дальнейшему усилению защиты	45
Заключение.....	47
Список использованных источников.....	48

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

3НФ – Третья нормальная форма

БД – База данных

ИС – Информационная система

ПО – Программное обеспечение

СУБД – Система управления базами данных

ACID (Atomicity, Consistency, Isolation, Durability) – Набор требований к транзакционной системе (Атомарность, Согласованность, Изолированность, Долговечность)

AES (Advanced Encryption Standard) – Симметричный алгоритм блочного шифрования

API (Application Programming Interface) – Программный интерфейс приложения

DDL (Data Definition Language) – Язык определения данных (команды создания и изменения структуры БД)

DML (Data Manipulation Language) – Язык манипулирования данными (команды вставки, обновления, удаления)

ER-диаграмма (Entity-Relationship Diagram) – Диаграмма «сущность-связь»

HTTP (HyperText Transfer Protocol) – Протокол передачи гипертекста

JWT (JSON Web Token) – Веб-токен JSON для безопасной передачи данных между сторонами

ORM (Object-Relational Mapping) – Технология связывания данных из БД с объектами языка программирования

PITR (Point-in-Time Recovery) – Восстановление на определенный момент времени

RBAC (Role-Based Access Control) – Ролевая модель управления доступом

SIEM (Security Information and Event Management) – Система управления событиями информационной безопасности

SQL (Structured Query Language) – Язык структурированных запросов

TLS/SSL (Transport Layer Security / Secure Sockets Layer) – Протоколы криптографической защиты передаваемой информации

WAL (Write-Ahead Logging) – Журнал предзаписи (механизм обеспечения целостности в СУБД)

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Авторизация – процесс предоставления пользователю определенных прав доступа к ресурсам системы после успешной аутентификации.

Аутентификация – процедура проверки подлинности пользователя путем сравнения введенных им учетных данных с данными, хранящимися в системе.

Инфологическая модель – концептуальное представление предметной области, описывающее сущности, их атрибуты и взаимосвязи, независимое от конкретной реализации СУБД.

Кикшеринг – система краткосрочной аренды самокатов и электросамокатов по принципу самообслуживания.

Ключ шифрования – секретная информация, используемая криптографическим алгоритмом при шифровании и дешифровании сообщений.

Нормализация – процесс организации данных в базе данных с целью устранения избыточности и потенциальных аномалий при их обработке.

Представление (View) – виртуальная таблица, содержимое которой определяется запросом к одной или нескольким базовым таблицам.

Принцип минимальных привилегий – концепция безопасности, согласно которой пользователю предоставляются только те права доступа, которые необходимы для выполнения его задач.

Резервное копирование – процесс создания копии данных на носителе, предназначенном для восстановления данных в случае их повреждения или утраты.

Ролевая модель доступа (RBAC) – политика управления доступом, при которой права на выполнение операций назначаются не конкретным пользователям, а ролям.

SQL-инъекция – вид атаки на базу данных, заключающийся во внедрении произвольного SQL-кода в запрос через поля ввода приложения.

Транзакция – группа последовательных операций с базой данных, которая представляет собой логическую единицу работы с данными.

Триггер – хранимая в базе данных процедура, которая автоматически выполняется при наступлении определенного события (например, вставки, обновления или удаления данных) в указанной таблице.

Хэширование – преобразование входного массива данных произвольной длины в выходную битовую строку фиксированной длины.

ВВЕДЕНИЕ

Актуальность данной работы заключается в том, что в последние годы сервисы краткосрочной аренды средств индивидуальной мобильности, такие как прокат электросамокатов (кикшеринг), приобрели широкую популярность в городских агломерациях. Функционирование таких систем неразрывно связано с обработкой больших объемов критически важных данных, включая персональную информацию пользователей, платежные реквизиты, а также данные о геолокации в реальном времени. В подобных условиях обеспечение информационной безопасности баз данных становится первостепенной задачей, так как утечка или компрометация этих сведений может привести к серьезным финансовым и репутационным потерям.

Целью данной курсовой работы является комплексное проектирование, реализация и обеспечение безопасности базы данных для информационной системы «Прокат электросамокатов», а также создание защищенного сервиса для взаимодействия с ней.

Для достижения поставленной цели были определены следующие **задачи**:

- Провести системный анализ предметной области, спроектировать и нормализовать инфологическую модель данных.
- Выполнить физическую реализацию реляционной базы данных в среде СУБД PostgreSQL.
- Внедрить многоуровневую систему защиты, включающую мониторинг операций, шифрование конфиденциальных данных и ролевую модель разграничения доступа.
- Разработать серверное приложение (API), обеспечивающее безопасное взаимодействие с базой данных с использованием современных механизмов аутентификации.
- Реализовать и протестировать процедуры резервного копирования и восстановления данных для обеспечения их доступности и целостности.
- Провести итоговый аудит безопасности реализованного решения, выявить его сильные стороны и определить пути дальнейшего совершенствования.
- Объектом исследования являются процессы хранения, обработки и защиты данных в информационных системах кикшеринга. Предметом исследования выступают методы и средства обеспечения безопасности баз данных на платформе PostgreSQL и связанных с ними сервисов.

Работа состоит из введения, шести основных разделов, заключения, списка использованных источников и приложений. В первом разделе рассматривается процесс инфологического моделирования. Второй раздел посвящен физической реализации БД. Третий раздел описывает внедрение механизмов безопасности. Четвертый и пятый разделы детализируют разработку API-сервиса и настройку резервного копирования. В шестом разделе проводится комплексный аудит безопасности. В заключении подводятся итоги проделанной работы.

1 СИСТЕМНЫЙ АНАЛИЗ ИНФОРМАЦИОННОЙ СИСТЕМЫ «ПРОКАТ ЭЛЕКТРОСАМОКАТОВ»

Объектом проектирования является база данных для информационной системы (ИС) краткосрочной аренды электросамокатов (кикшеринга). Данная предметная область характеризуется высокой динамикой изменения данных (геолокация, статус аренды) и требованиями к точности финансовых транзакций.

1.1 Описание процессов и задач

Разрабатываемая база данных выступает ядром информационной системы и обеспечивает автоматизацию следующих ключевых бизнес-процессов:

- **Управление парком электросамокатов:** отслеживание местоположения, технического состояния и уровня заряда каждого самоката в режиме реального времени.
- **Взаимодействие с пользователями:** регистрация новых клиентов, управление их данными и платежной информацией.
- **Процесс аренды:** предоставление пользователям возможности находить, бронировать, начинать и завершать поездки.
- **Финансовые операции:** автоматический расчет стоимости поездок на основе тарифов, обработка платежей и управление промокодами.
- **Техническое обслуживание:** планирование и учет работ по ремонту, замене аккумуляторов и перемещению самокатов.

Источники данных:

Функционирование системы обеспечивается поступлением данных из следующих источников:

- **IoT-модули на электросамокатах:** GPS-данные о местоположении, информация об уровне заряда батареи, данные о состоянии (заблокирован/в движении). Информация поступает непрерывно в реальном времени.
- **Мобильное приложение пользователя:** данные при регистрации (ФИО, телефон, email), платежная информация, данные сканирования QR-кода для начала поездки, отзывы и обращения в поддержку.
- **Внутренняя система администрирования:** информация о тарифах, промокодах, зонах для парковки, а также данные, вносимые сотрудниками сервиса (например, отчеты о ремонте).

Потребители информации:

Основными потребителями информации из базы данных являются:

- **Пользователи (клиенты сервиса):** используют мобильное приложение для поиска ближайшего самоката, просмотра информации о нем (уровень заряда, тариф), начала и завершения аренды, а также для просмотра истории своих поездок. Информация представляется в виде интерактивной карты, карточек самокатов и списков поездок.
- **Технический специалист (сотрудник сервиса):** получает информацию о самокатах, требующих обслуживания (низкий заряд, поломка), их местонахождении. Также отслеживает самокаты, оставленные вне разрешенных зон. Информация представляется в виде списка задач на специальном интерфейсе, отсортированного по приоритету, и на карте города.

Ограничения предметной области:

В ходе анализа были выявлены следующие бизнес-правила, накладывающие ограничения на структуру данных:

- Один пользователь не может одновременно иметь более одной активной поездки. Связь между пользователем и его текущей поездкой должна быть эксклюзивной.
- Электросамокат со статусом в ремонте или с критически низким уровнем заряда не может быть использован для новой поездки.
- Каждая поездка должна быть обязательно связана с одним пользователем и одним самокатом. Не может существовать поездок «в никуда».
- Завершение поездки возможно только в пределах разрешенной географической зоны.

1.2 Выделение сущностей и построение ER-диаграммы

На основе проведенного системного анализа были выделены следующие информационные объекты (сущности):

- **Пользователь (User):** Хранит информацию о клиентах сервиса. Каждый экземпляр – это уникальный зарегистрированный пользователь.
- **Электросамокат (Scooter):** Моделирует каждый отдельный самокат в парке компании. Каждый экземпляр – физический самокат с уникальным номером.
- **Поездка (Trip):** Фиксирует факт аренды самоката конкретным пользователем. Каждый экземпляр – это одна завершенная или текущая поездка.
- **Платеж (Payment):** Содержит информацию о финансовых транзакциях за поездки.

– **Зона (Zone):** Описывает географические области, где разрешено использование и парковка самокатов.

– **Техобслуживание (Maintenance):** Учитывает все сервисные работы, проводимые с самокатами.

– **Тариф (Tariff):** Определяет стоимость использования сервиса.

Анализ связей:

Между выделенными сущностями установлены следующие отношения:

– Между сущностями «Пользователь» и «Поездка» существует связь «совершает» степени 1:M (один ко многим). Один пользователь может совершить много поездок, но каждая поездка совершается только одним пользователем.

– Между сущностями «Электросамокат» и «Поездка» существует связь «используется в» степени 1:M. Один самокат может быть использован во многих поездках, но в каждой поездке используется только один самокат.

– Между сущностями «Поездка» и «Платеж» существует связь «оплачивается» степени 1:M. Каждая поездка связана с одним или несколькими платежами, а каждый платеж относится к одной конкретной поездке.

– Между сущностями «Электросамокат» и «Техобслуживание» существует связь «проходит» степени 1:M. Один самокат может проходить техобслуживание много раз, и каждая запись о техобслуживании относится к одному самокату.

– Между сущностями «Электросамокат» и «Зона» существует связь «находится в» степени M:1. Один самокат в данный момент времени находится в одной зоне, но в одной зоне может находиться много самокатов.

– Между сущностями «Поездка» и «Тариф» существует связь «тарифицируется по» степени M:1. Каждая поездка рассчитывается по одному тарифу, но один тариф может применяться ко многим поездкам.

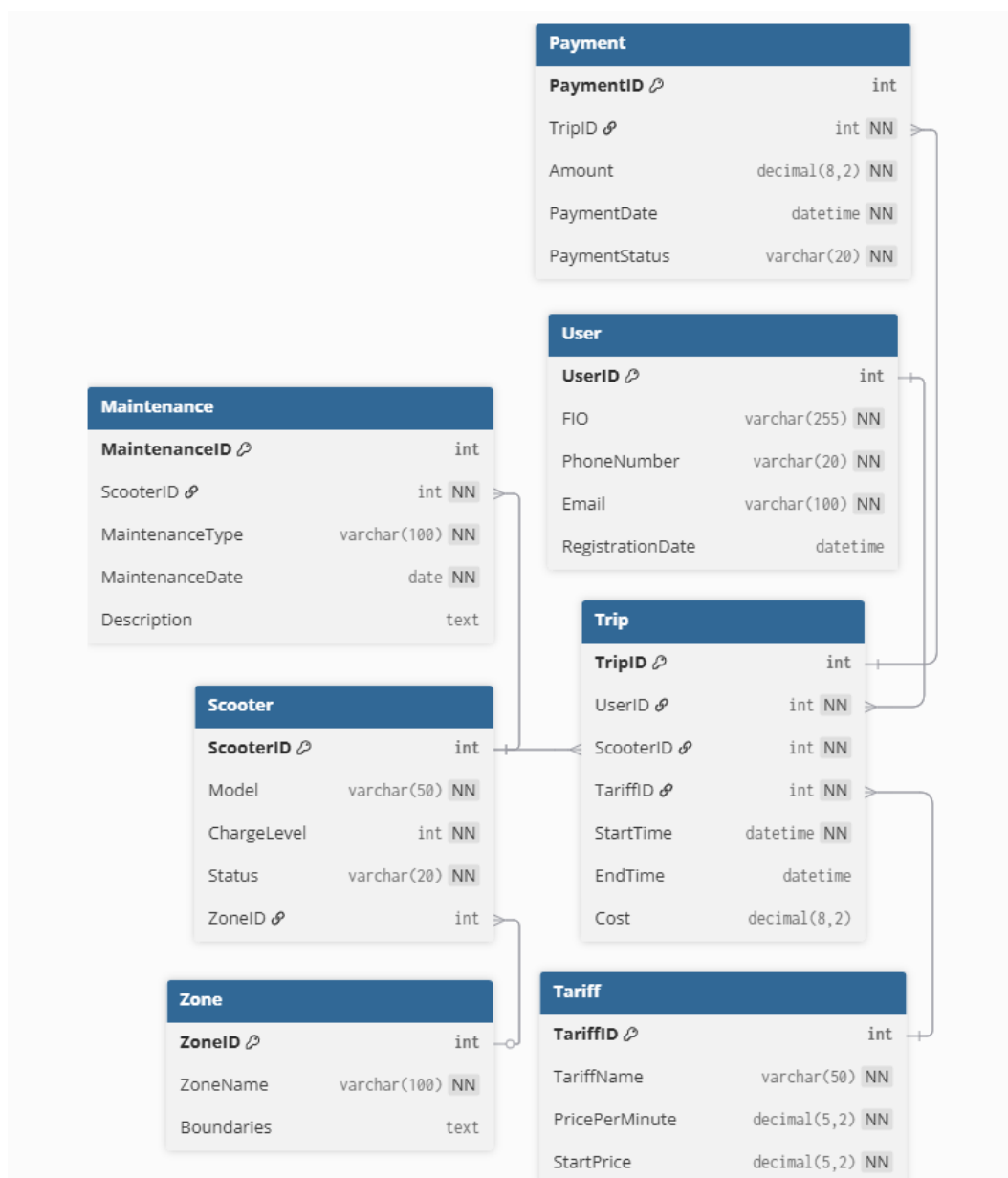


Рисунок 1 – ER-диаграмма

На основе ER-диаграммы сформирована предварительная схема отношений. Первичные ключи (PK) выделены подчеркиванием, внешние ключи (FK) — *курсивом*.

- **User** (UserID, FIO, PhoneNumber, Email, RegistrationDate)
- **Zone** (ZoneID, ZoneName, Boundaries)
- **Tariff** (TariffID, TariffName, PricePerMinute, StartPrice)
- **Scooter** (ScooterID, Model, ChargeLevel, Status, *ZoneID*)
- **Maintenance** (MaintenanceID, *ScooterID*, MaintenanceType, MaintenanceDate, Description)
- **Trip** (TripID, *UserID*, *ScooterID*, *TariffID*, StartTime, EndTime, Cost)
- **Payment** (PaymentID, *TripID*, Amount, PaymentDate, PaymentStatus)

1.3 Приведение схемы отношений к третьей нормальной форме

Для обеспечения целостности данных и устранения избыточности была проведена процедура нормализации спроектированной схемы отношений.

Нормализация – это процесс организации данных в базе данных, который включает создание таблиц и установление отношений между ними в соответствии с правилами, предназначенными как для защиты данных, так и для повышения гибкости базы данных путем устранения избыточности и несогласованных зависимостей. Проведем нормализацию последовательно.

Отношение находится в первой нормальной форме, если все его атрибуты являются атомарными, то есть содержат только одно значение.

Анализ на соответствие 1НФ:

- **User:** Атрибуты FIO, PhoneNumber, Email являются атомарными, так как для целей системы не требуется их дальнейшее разделение (например, поиск по фамилии отдельно от имени).
- **Scooter:** Все атрибуты (Model, ChargeLevel, Status) содержат простые значения.
- **Trip:** Атрибуты StartTime, EndTime, Cost являются неделимыми.
- **Payment, Zone, Maintenance, Tariff:** Аналогично, все атрибуты в данных отношениях являются атомарными.

Все отношения, представленные в схеме, содержат только простые атрибуты. Таким образом, вся схема отношений находится в 1НФ.

Отношение находится во второй нормальной форме, если оно находится в 1НФ и каждый неключевой атрибут функционально полно зависит от первичного ключа. Это правило актуально для отношений с составными первичными ключами.

Анализ на соответствие 2НФ:

- В каждом из отношений (User, Scooter, Trip, Payment, Zone, Maintenance, Tariff) первичный ключ является простым, то есть состоит из одного атрибута (UserID, ScooterID и т.д.).
- Поскольку первичные ключи не являются составными, не может существовать частичной зависимости неключевого атрибута от части ключа. Любой неключевой атрибут по определению может зависеть только от всего простого ключа целиком.

Так как все отношения находятся в 1НФ и имеют простые первичные ключи, все они автоматически удовлетворяют требованиям 2НФ.

Отношение находится в третьей нормальной форме, если оно находится во 2НФ и в нем отсутствуют транзитивные зависимости (когда неключевой атрибут зависит от другого неключевого атрибута).

Анализ на соответствие 3НФ:

- **User:** Атрибуты FIO, PhoneNumber, Email зависят только от UserID, а не друг от друга. Транзитивные зависимости отсутствуют.
- **Scooter:** Model, ChargeLevel, Status зависят от ScooterID. ZoneID также является атрибутом, напрямую зависящим от самоката. Зависимостей между неключевыми атрибутами нет.
- **Trip:** StartTime, EndTime, Cost зависят только от уникального идентификатора поездки TripID.
- **Tariff:** TariffName, PricePerMinute, StartPrice зависят от TariffID. Может показаться, что PricePerMinute зависит от TariffName, но оба этих атрибута напрямую определяются уникальным TariffID, поэтому транзитивной зависимости нет.
- **Payment, Zone, Maintenance:** Аналогично, все неключевые атрибуты в этих таблицах напрямую зависят только от их первичных ключей.

Все отношения находятся во 2НФ и не содержат транзитивных зависимостей неключевых атрибутов от первичного ключа. Следовательно, вся схема отношений находится в 3НФ и является нормализованной.

1.4 Моделирование уровня представлений

Потребитель «Пользователи»

Представление 1. «История моих поездок»

Цель: Предоставить пользователю краткую информацию о его прошлых поездках.

Атрибуты:

- Дата поездки (из Trip)
- Модель самоката (из Scooter)
- Длительность поездки (вычисляется на основе StartTime и EndTime из Trip)
- Стоимость (из Trip)

- Статус платежа (из Payment)

Представление 2. «Доступные самокаты»

Цель: Показать пользователю на карте доступные для аренды самокаты с ключевой информацией.

Атрибуты:

- ID самоката (из Scooter)
- Местоположение (из Scooter)
- Уровень заряда (из Scooter)
- Тариф (название тарифа из Tariff)
- Стоимость минуты (из Tariff)

Потребитель «Технический специалист»

Представление 3. «Самокаты для зарядки»

Цель: Сформировать список самокатов, требующих срочной замены аккумулятора.

Атрибуты:

- ID самоката (из Scooter)
- Модель (из Scooter)
- Текущий уровень заряда (из Scooter)
- Местоположение (из Scooter)
- Название зоны (из Zone)

Представление 4. «Журнал обслуживания самоката»

Цель: Показать полную историю технических работ по конкретному самокату.

Атрибуты:

- ID самоката (из Scooter)
- Модель (из Scooter)
- Дата обслуживания (из Maintenance)
- Тип работ (из Maintenance)
- Описание работ (из Maintenance)

2 РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ В РАМКАХ СУБД POSTGRESQL

В данном разделе описывается процесс физической реализации спроектированной модели данных. В качестве целевой платформы выбрана объектно-реляционная система управления базами данных (СУБД) PostgreSQL.

2.1 Обоснование выбора СУБД

Выбор PostgreSQL в качестве инструмента реализации обусловлен совокупностью технических характеристик, отвечающих требованиям разрабатываемой системы:

- **Надежность и соответствие стандартам (ACID):** Критически важно для систем, обрабатывающих финансовые транзакции и персональные данные.
- **Поддержка сложных типов данных:** Наличие встроенных типов для работы с геометрическими данными (PostGIS) и JSONB позволяет эффективно хранить координаты зон и логи изменений.
- **Развитые механизмы безопасности:** Встроенная поддержка ролевой модели доступа (RBAC), SSL-шифрования и аудита.
- **Лицензионная чистота:** PostgreSQL является свободным программным обеспечением (Open Source), что снижает затраты на внедрение.

Администрирование СУБД осуществлялось с использованием графического интерфейса pgAdmin 4.

2.2 Создание и наполнение таблиц

Первым шагом создадим новую базу данных с названием `scooter_rental_db` для нашего проекта.

```
CREATE DATABASE scooter_rental_db
WITH
OWNER = postgres
ENCODING = 'UTF8'
CONNECTION LIMIT = -1
IS_TEMPLATE = False;
```

Далее, на основе спроектированной схемы, были созданы таблицы с использованием DDL-команд.

Реализация таблиц сущностей:

Таблица для разрешенных географических зон

```
CREATE TABLE IF NOT EXISTS public.Zone (
    ZoneID serial PRIMARY KEY,
    ZoneName varchar(100) NOT NULL,
    Boundaries text -- Предполагается хранение в формате GeoJSON
);
```

Таблица тарифов на аренду

```
CREATE TABLE IF NOT EXISTS public.Tariff (
    TariffID serial PRIMARY KEY,
    TariffName varchar(50) NOT NULL,
    PricePerMinute decimal(5, 2) NOT NULL,
    StartPrice decimal(5, 2) NOT NULL DEFAULT 0.00
);
```

Таблица пользователей (клиентов) сервиса

```
CREATE TABLE IF NOT EXISTS public."User" (
    UserID serial PRIMARY KEY,
    FIO varchar(255) NOT NULL,
    PhoneNumber varchar(20) UNIQUE NOT NULL,
    Email varchar(100) UNIQUE NOT NULL,
    RegistrationDate timestamp NOT NULL DEFAULT now()
);
```

Таблица для хранения информации об электросамокатах

```
CREATE TABLE IF NOT EXISTS public.Scooter (
    ScooterID serial PRIMARY KEY,
    Model varchar(50) NOT NULL,
    ChargeLevel int NOT NULL,
    Status varchar(20) NOT NULL DEFAULT 'available',
    ZoneID int,
    CONSTRAINT fk_zone
        FOREIGN KEY(ZoneID)
        REFERENCES Zone(ZoneID)
);
```

Таблица для учета работ по техническому обслуживанию

```
CREATE TABLE IF NOT EXISTS public.Maintenance (
    MaintenanceID serial PRIMARY KEY,
    ScooterID int NOT NULL,
    MaintenanceType varchar(100) NOT NULL,
```

```

MaintenanceDate date NOT NULL,
Description text,
CONSTRAINT fk_scooter
    FOREIGN KEY(ScooterID)
    REFERENCES Scooter(ScooterID)
);

```

Таблица поездок, фиксирующая каждую аренду

```

CREATE TABLE IF NOT EXISTS public.Trip (
    TripID serial PRIMARY KEY,
    UserID int NOT NULL,
    ScooterID int NOT NULL,
    TariffID int NOT NULL,
    StartTime timestamp NOT NULL,
    EndTime timestamp,
    Cost decimal(8, 2),
    CONSTRAINT fk_user
        FOREIGN KEY(UserID)
        REFERENCES "User"(UserID),
    CONSTRAINT fk_scooter
        FOREIGN KEY(ScooterID)
        REFERENCES Scooter(ScooterID),
    CONSTRAINT fk_tariff
        FOREIGN KEY(TariffID)
        REFERENCES Tariff(TariffID)
);

```

Таблица платежей по поездкам

```

CREATE TABLE IF NOT EXISTS public.Payment (
    PaymentID serial PRIMARY KEY,
    TripID int NOT NULL,
    Amount decimal(8, 2) NOT NULL,
    PaymentDate timestamp NOT NULL DEFAULT now(),
    PaymentStatus varchar(20) NOT NULL DEFAULT 'completed',
    CONSTRAINT fk_trip
        FOREIGN KEY(TripID)
        REFERENCES Trip(TripID)
);

```

После создания структуры таблицы были наполнены тестовыми данными (DML-операции INSERT). В каждую таблицу добавлено не менее 8 записей для обеспечения репрезентативности выборки при тестировании.

Заполнение таблицы Zone

```
-- С упрощенными полигонами
INSERT INTO public.Zone (ZoneName, Boundaries) VALUES
('Центральный район', '{"type": "Polygon", "coordinates": [[[30.35, 59.93],
    [30.35, 59.94], [30.38, 59.94], [30.38, 59.93], [30.35, 59.93]]]}'),
('Адмиралтейский район', '{"type": "Polygon", "coordinates": [[[30.28, 59.91],
    [30.28, 59.92], [30.31, 59.92], [30.31, 59.91], [30.28, 59.91]]]}'),
('Петроградский район', '{"type": "Polygon", "coordinates": [[[30.28, 59.96],
    [30.28, 59.97], [30.30, 59.97], [30.30, 59.96], [30.28, 59.96]]]}');
```

Заполнение таблицы Tariff

```
INSERT INTO public.Tariff (TariffName, PricePerMinute, StartPrice) VALUES
('Стандарт', 5.00, 50.00),
('Эко', 4.50, 30.00),
('Премиум', 7.50, 70.00);
```

Заполнение таблицы "User"

```
INSERT INTO public."User" (FIO, PhoneNumber, Email) VALUES
('Иванов Иван Иванович', '+79211234567', 'ivanov@email.com'),
('Петрова Мария Сергеевна', '+79119876543', 'petrova@email.com'),
('Сидоров Алексей Петрович', '+79055554433', 'sidorov@email.com'),
('Смирнова Ольга Викторовна', '+79998887766', 'smirnova@email.com'),
('Кузнецов Дмитрий Андреевич', '+79213332211', 'kuznetsov@email.com'),
('Васильева Анна Михайловна', '+79114445566', 'vasilieva@email.com'),
('Михайлов Сергей Александрович', '+79067778899', 'mikhailov@email.com'),
('Новикова Екатерина Дмитриевна', '+79211110022', 'novikova@email.com');
```

Заполнение таблицы Scooter

```
INSERT INTO public.Scooter (Model, ChargeLevel, Status, ZoneID) VALUES
('Ninebot G30', 85, 'available', 1),
('Kugoo M4', 100, 'available', 1),
('Xiaomi M365', 45, 'available', 2),
('Ninebot G30', 25, 'low_battery', 3),
('Xiaomi M365', 95, 'in_use', 1),
('Kugoo M4', 70, 'available', 2),
('Ninebot G30', 15, 'maintenance', 3),
```

```
('Xiaomi M365', 80, 'available', 1);
```

Заполнение таблицы Maintenance

```
INSERT INTO public.Maintenance (ScooterID, MaintenanceType, MaintenanceDate,
    Description) VALUES
(7, 'Замена тормозного диска', '2025-09-27', 'Плановая замена'),
(4, 'Зарядка аккумулятора', '2025-09-28', 'Низкий уровень заряда'),
(1, 'Проверка давления в шинах', '2025-09-25', 'Плановое ТО');
```

Заполнение таблицы Trip

```
INSERT INTO public.Trip (UserID, ScooterID, TariffID, StartTime, EndTime, Cost)
VALUES
(1, 5, 1, '2025-09-28 10:00:00', '2025-09-28 10:15:00', 125.00),
(2, 3, 2, '2025-09-28 11:30:00', '2025-09-28 11:50:00', 120.00),
(3, 1, 1, '2025-09-27 15:00:00', '2025-09-27 15:30:00', 200.00),
(1, 2, 3, '2025-09-27 18:20:00', '2025-09-27 18:30:00', 145.00),
(4, 6, 1, '2025-09-28 09:00:00', '2025-09-28 09:12:00', 110.00),
(5, 8, 2, '2025-09-28 12:00:00', '2025-09-28 12:25:00', 142.50),
(2, 1, 1, '2025-09-28 13:00:00', '2025-09-28 13:05:00', 75.00),
(6, 2, 1, '2025-09-28 14:10:00', '2025-09-28 14:40:00', 200.00);
```

Заполнение таблицы Payment

```
INSERT INTO public.Payment (TripID, Amount, PaymentStatus) VALUES
(1, 125.00, 'completed'),
(2, 120.00, 'completed'),
(3, 200.00, 'completed'),
(4, 145.00, 'completed'),
(5, 110.00, 'completed'),
(6, 142.50, 'completed'),
(7, 75.00, 'completed'),
(8, 200.00, 'completed');
```

Дополнительно была произведена модификация структуры таблицы User для повышения безопасности: добавлен атрибут LastLogin для отслеживания активности.

```
ALTER TABLE public."User"
ADD COLUMN LastLogin timestamp;
```

2.3 Реализация индексов и внешних ключей

Для ускорения выполнения запросов, особенно операций JOIN и WHERE, добавим индексы на внешние ключи и часто используемые для поиска поля.

Взаимосвязи между таблицами в реляционной модели данных реализованы с помощью механизма внешних ключей. Этот механизм обеспечивает ссылочную целостность данных, гарантируя, что связанные данные остаются согласованными. Например, невозможно создать запись о поездке для пользователя, которого не существует в системе.

В разработанной базе данных `scooter_rental_db` были установлены следующие ключевые взаимосвязи при создании таблиц:

- **Scooter → Zone (1:N):** Каждый самокат в определенный момент времени находится в одной географической зоне. Связь установлена через внешний ключ `Scooter.ZoneID`, ссылающийся на `Zone.ZoneID`.
- **Maintenance → Scooter (1:N):** Для одного самоката может быть множество записей о его техническом обслуживании. Связь установлена через внешний ключ `Maintenance.ScooterID`, ссылающийся на `Scooter.ScooterID`.
- **Trip → User (1:N):** Один пользователь может совершить множество поездок. Связь установлена через внешний ключ `Trip.UserID`, ссылающийся на `"User".UserID`.
- **Trip → Scooter (1:N):** Один самокат может быть использован во множестве поездок. Связь установлена через внешний ключ `Trip.ScooterID`, ссылающийся на `Scooter.ScooterID`.
- **Trip → Tariff (1:N):** Множество поездок могут тарифицироваться по одному тарифу. Связь установлена через внешний ключ `Trip.TariffID`, ссылающийся на `Tariff.TariffID`.
- **Payment → Trip (1:N):** Одна поездка может иметь один или несколько связанных платежей. Связь установлена через внешний ключ `Payment.TripID`, ссылающийся на `Trip.TripID`.

Все эти ограничения были определены с помощью конструкции `CONSTRAINT FOREIGN KEY` в SQL-скриптах создания таблиц, представленных в разделе **Ошибка! Источник ссылки не найден.**

Индексы для таблицы `Scooter`

```
CREATE INDEX idx_scooter_zoneid ON public.Scooter (ZoneID);
```

```
CREATE INDEX idx_scooter_status ON public.Scooter(Status);
```

Индексы для таблицы Maintenance

```
CREATE INDEX idx_maintenance_scooterid ON public.Maintenance(ScooterID);
```

Индексы для таблицы Trip

```
CREATE INDEX idx_trip_userid ON public.Trip(UserID);
```

```
CREATE INDEX idx_trip_scooterid ON public.Trip(ScooterID);
```

```
CREATE INDEX idx_trip_tariffid ON public.Trip(TariffID);
```

Индексы для таблицы Payment

```
CREATE INDEX idx_payment_tripid ON public.Payment(TripID);
```

2.4 Создание представлений

Создадим представления, спроектированные в первом разделе, для упрощения доступа к данным для разных классов потребителей.

Потребитель «Пользователи»

Представление 1. «История моих поездок»

```
CREATE VIEW user_trip_history AS
SELECT
    t.TripID,
    t.UserID,
    t.StartTime AS "Дата поездки",
    s.Model AS "Модель самоката",
    (t.EndTime - t.StartTime) AS "Длительность",
    t.Cost AS "Стоимость",
    p.PaymentStatus AS "Статус платежа"
FROM Trip t
JOIN Scooter s ON t.ScooterID = s.ScooterID
JOIN Payment p ON t.TripID = p.TripID
```

Представление 2. «Доступные самокаты»

```
CREATE VIEW available_scooters AS
SELECT
    s.ScooterID,
    z.ZoneName AS "Местоположение",
    s.ChargeLevel AS "Уровень заряда",
    tr.TariffName AS "Тариф",
```



```

        tr.PricePerMinute AS "Стоимость минуты"
FROM Scooter s
JOIN Zone z ON s.ZoneID = z.ZoneID
-- Присоединяем тариф через таблицу поездок
-- (представим, что все самокаты в зоне работают по первому тарифу)
JOIN Tariff tr ON tr.TariffID = 1
WHERE s.Status = 'available';

```

Потребитель «Технический специалист»

Представление 3. «Самокаты для зарядки»

```

CREATE VIEW scooters_for_charging AS
SELECT
    s.ScooterID,
    s.Model,
    s.ChargeLevel AS "Текущий уровень заряда",
    z.ZoneName AS "Название зоны"
FROM Scooter s
JOIN Zone z ON s.ZoneID = z.ZoneID
WHERE s.Status = 'available' AND s.ChargeLevel < 30
ORDER BY s.ChargeLevel ASC;

```

Представление 4. «Журнал обслуживания самоката»

```

CREATE VIEW maintenance_log AS
SELECT
    s.ScooterID,
    s.Model,
    m.MaintenanceDate AS "Дата обслуживания",
    m.MaintenanceType AS "Тип работ",
    m.Description AS "Описание работ"
FROM Maintenance m
JOIN Scooter s ON m.ScooterID = s.ScooterID
ORDER BY s.ScooterID, m.MaintenanceDate DESC;

```

2.5 Тестирование базы данных

Для проверки корректности созданной схемы, целостности данных и работоспособности бизнес-логики (реализованной в представлениях) были написаны автоматизированные тесты с использованием фреймворка `pytest` и библиотеки `psycopg2` для подключения к PostgreSQL.

Для запуска тестов необходимо установить следующие Python-библиотеки:

```
pip install pytest
pip install psycopg2-binary
```

Тесты размещены в файле `test_db.py` (**Ошибка! Источник ссылки не найден.**).

Используется механизм фикстур `pytest` для управления подключением к базе данных.

Запрос: `SELECT COUNT(*) FROM public."User";`

Ожидаемый результат: Запрос возвращает число, равное или большее 8, что подтверждает корректное выполнение скрипта для заполнения таблицы исходными данными.

Запрос: `SELECT "Текущий уровень заряда" FROM public.scooters_for_charging;`

Ожидаемый результат: Все числовые значения, возвращенные этим запросом, должны быть строго меньше 30. Это доказывает, что логический фильтр (`WHERE ChargeLevel < 30`) в представлении работает корректно.

Действие: Попытка выполнить SQL-запрос `INSERT INTO public.Trip (UserID, ScooterID, TariffID, StartTime) VALUES (999999, 1, 1, NOW());`, где `UserID = 999999` заведомо не существует в таблице `"User"`.

Ожидаемый результат: СУБД должна прервать выполнение запроса и сгенерировать ошибку нарушения целостности внешнего ключа (`ForeignKeyViolation`). Запись не должна быть добавлена в таблицу.

Действие: Попытка выполнить SQL-запрос `INSERT INTO public."User" (FIO, PhoneNumber, Email) VALUES ('Тестовый Пользователь', '+700000000000', 'ivanov@email.com');`, где `Email = 'ivanov@email.com'` уже существует в таблице.

Ожидаемый результат: СУБД должна прервать выполнение запроса и сгенерировать ошибку нарушения уникальности (`UniqueViolation`). Новая запись не должна быть добавлена в таблицу.

Действие: Выполняется `INSERT` в таблицу `Scooter`, намеренно пропуская столбец `Status`, для которого в схеме определено значение `DEFAULT 'available'`. Сразу после этого выполняется `SELECT Status` для только что вставленной записи.

Ожидаемый результат: Запрос на вставку выполняется успешно. Последующий `SELECT` возвращает значение `'available'`, что подтверждает корректную работу механизма `DEFAULT` в СУБД.

Для запуска тестов достаточно выполнить в терминале, находясь в каталоге с файлом `test_db.py`, команду:

```
pytest -v
```

```
C:\Users\muham\Desktop\itmo\_Studying(5th_sem)\DatabaseSecurity(2nd)\np2>pytest -v
C:\Users\muham\AppData\Local\Programs\Python\Python313\Lib\site-packages\pytest_asyncio\plugin.py:252: PytestDeprecationWarning: The configuration option "asyncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyncio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture loop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "package", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))

===== test session starts =====
platform win32 -- Python 3.13.2, pytest-8.3.5, pluggy-1.5.0 -- C:\Users\muham\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\muham\Desktop\itmo\_Studying(5th_sem)\DatabaseSecurity(2nd)\np2
plugins: anyio-4.8.0, locust-2.40.1, asyncio-1.2.0, cov-6.2.1, mock-3.15.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 6 items

test_db.py::test_connection_is_successful PASSED [ 16%]
test_db.py::test_users_table_is_populated PASSED [ 33%]
test_db.py::test_scooters_for_charging_view_logic PASSED [ 50%]
test_db.py::test_foreign_key_constraint PASSED [ 66%]
test_db.py::test_unique_email_constraint PASSED [ 83%]
test_db.py::test_scooter_status_default_value PASSED [100%]

===== 6 passed in 0.08s =====
```

Рисунок 2 – Результаты автоматизированного тестирования (pytest)

Таким образом, физическая реализация базы данных полностью соответствует спроектированной модели, обеспечивает целостность данных и готова к внедрению механизмов защиты.

3 ОБЕСПЕЧЕНИЕ БЕЗОПАСНОСТИ БАЗЫ ДАННЫХ

В данном разделе описывается комплекс мер по обеспечению информационной безопасности разработанной базы данных, включающий систему аудита событий, криптографическую защиту конфиденциальной информации и ролевую модель управления доступом.

3.1 Мониторинг операций в БД на основе триггеров

Для отслеживания всех изменений в данных (операций INSERT, UPDATE, DELETE) будет создана система аудита на основе триггеров.

Первым шагом создадим универсальную таблицу `action_log` для хранения записей аудита. Она будет фиксировать тип операции, время, пользователя, название таблицы и сами измененные данные.

```
CREATE TABLE public.action_log (  
    log_id serial PRIMARY KEY,  
    operation_type char(1) NOT NULL,  
    operation_time timestamp NOT NULL DEFAULT now(),  
    db_user text NOT NULL DEFAULT current_user,  
    table_name text NOT NULL,  
    old_data jsonb,  
    new_data jsonb  
);  
  
COMMENT ON COLUMN public.action_log.operation_type IS 'I - Insert, U - Update,  
    D - Delete';
```

Далее создадим триггерную функцию `log_action()`, которая будет вызываться при любом изменении данных и записывать соответствующую информацию в `action_log`.

```
CREATE OR REPLACE FUNCTION log_action() RETURNS TRIGGER AS $$  
BEGIN  
    IF (TG_OP = 'INSERT') THEN  
        INSERT INTO public.action_log (operation_type, table_name, new_data)  
        VALUES ('I', TG_TABLE_NAME, to_jsonb(NEW));  
    ELSIF (TG_OP = 'UPDATE') THEN  
        INSERT INTO public.action_log (operation_type, table_name, old_data,  
        new_data)  
        VALUES ('U', TG_TABLE_NAME, to_jsonb(OLD), to_jsonb(NEW));  
    END IF;  
END;
```

```

ELSIF (TG_OP = 'DELETE') THEN
    INSERT INTO public.action_log (operation_type, table_name, old_data)
    VALUES ('D', TG_TABLE_NAME, to_jsonb(OLD));
END IF;
RETURN NULL; -- Результат не важен для AFTER-триггера
END;
$$ LANGUAGE plpgsql;

```

Теперь применим созданную функцию в качестве триггера для каждой основной таблицы в нашей базе данных: "User", Scooter, Trip, Payment и Maintenance.

```

CREATE TRIGGER user_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public."User"
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER scooter_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Scooter
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER trip_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Trip
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER payment_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Payment
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER maintenance_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Maintenance
FOR EACH ROW EXECUTE FUNCTION log_action();

```

Демонстрация работы системы аудита:

```

INSERT INTO public."User" (FIO, PhoneNumber, Email)
VALUES ('Тестов Тест Тестович', '+79001234567', 'test@test.com');

SELECT operation_type, operation_time, db_user, table_name, new_data->>'fio' as
    fio
FROM public.action_log;

```

Data Output Messages Notifications					
	operation_type character (1)	operation_time timestamp without time zone	db_user text	table_name text	fio text
1	I	2025-09-29 12:16:09.172609	postgres	User	'Гбв@ў 'Гбв 'Гбв@ўЕз
2	D	2025-09-29 12:33:22.248565	postgres	User	[null]
3	I	2025-09-29 12:34:10.378005	postgres	User	Тестов Тест Тестович

Рисунок 3 – Логиrowание операции INSERT в таблицу "User"

```
UPDATE public.Scooter SET Status = 'in_use', ChargeLevel = 80 WHERE ScooterID
= 1;

SELECT operation_type, operation_time, table_name, old_data->>'status' as
old_status, new_data->>'status' as new_status
FROM public.action_log WHERE operation_type = 'U';
```

Data Output Messages Notifications					
	operation_type character (1)	operation_time timestamp without time zone	table_name text	old_status text	new_status text
1	U	2025-09-29 12:36:05.041405	scooter	available	in_use

Рисунок 4 – Логиrowание операции UPDATE в таблице "Scooter"

```
DELETE FROM public.Maintenance WHERE MaintenanceID = 3;

SELECT operation_type, operation_time, table_name, old_data->>'description' as
deleted_description
FROM public.action_log WHERE operation_type = 'D';
```

Data Output Messages Notifications				
	operation_type character (1)	operation_time timestamp without time zone	table_name text	deleted_description text
1	D	2025-09-29 12:33:22.248565	User	[null]
2	D	2025-09-29 12:36:59.752963	maintenance	Плановое ТО

Рисунок 5 – Логиrowание операции DELETE из таблицы Maintenance (и "User")

Система логирования успешно фиксирует все типы изменений в отслеживаемых таблицах.

3.2 Шифрование конфиденциальных данных

Для защиты особо чувствительных данных, таких как токены доступа или API-ключи, используем встроенные криптографические функции PostgreSQL.

Сначала необходимо убедиться, что в базе данных активировано расширение pgcrypto.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Создадим таблицу `user_api_keys` для хранения API-ключей пользователей.

```
CREATE TABLE public.user_api_keys (  
    key_id serial PRIMARY KEY,  
    user_id int NOT NULL REFERENCES public."User" (UserID),  
    api_key bytea NOT NULL,  
    created_at timestamp NOT NULL DEFAULT now()  
);
```

Зашифруем данные при вставке. В качестве пароля для шифрования будем использовать индивидуальный пароль суперпользователя: `!super_secret_kickscooter_pwd_2025`. Этот пароль передается в функцию шифрования напрямую, что гарантирует, что сам ключ не хранится в базе данных.

```
INSERT INTO public.user_api_keys (user_id, api_key) VALUES  
(1, pgp_sym_encrypt(  
    'scooter_user_1_key_a1b2c3d4e5'::text,  
    '!super_secret_kickscooter_pwd_2025'::text  
)),  
(2, pgp_sym_encrypt(  
    'scooter_user_2_key_f6g7h8i9j0'::text,  
    '!super_secret_kickscooter_pwd_2025'::text  
));
```

При просмотре таблицы видны только зашифрованные бинарные данные.

```
SELECT * FROM public.user_api_keys;
```

```
scooter_rental_db=# SELECT * FROM public.user_api_keys;
key_id | user_id |                               | created_at | api_key
-----+-----+-----+-----+-----
1 | 1 | \xc30d0407030288a41180e8e93ade6ed24e0151396fc6c74e19309e18f40804c96cf20dd5785f53e86f19f58978474399e8388f4e008100da00273905fb7ff5589d5ee96a8f7d9b2cbffa34bd264b0cc9487024b0ae78d1a83e96011cf2cced | 2025-09-29 12:42:47.388711
2 | 2 | \xc30d0407030292e61d30f937403970d24e01107a5b2609793bf37c4a851f4764e64ea57e30ea8117d29015f1747ff6b6279646e1170a32e3d0d17da7426c8abf0245a3fcbc7953f61dbb912d92d7d7ae36dcf6b28eebb815a0d27c3b244a44 | 2025-09-29 12:42:47.388711
```

Рисунок 6 – Содержимое таблицы user_api_keys в зашифрованном виде

Даже имея полные права администратора (postgres), без знания исходного пароля невозможно прочитать данные.

```
SELECT
    key_id,
    user_id,
    pgp_sym_decrypt(api_key, 'wrong_password'::text) as decrypted_key
FROM public.user_api_keys
WHERE key_id = 1;
```

Data Output Messages Notifications

ERROR: Wrong key or corrupt data

ОШИБКА: Wrong key or corrupt data

SQL state: 39000

Рисунок 7 – Попытка доступа с неверным ключом

```
SELECT
    key_id,
    user_id,
    pgp_sym_decrypt(api_key, '!super_secret_kickscooter_pwd_2025'::text) as
    decrypted_key
FROM public.user_api_keys;
```

Data Output	Messages	Notifications
SQL		
key_id [PK] integer	user_id integer	decrypted_key text
1	1	scooter_user_1_key_a1b2c3d4e5
2	2	scooter_user_2_key_f6g7h8i9j0

Рисунок 8 – Успешная дешифровка корректным ключом

Таким образом, данные надежно защищены, и доступ к ним возможен только при наличии секретного пароля.

3.3 Разграничение доступа на основе ролевой модели (RBAC)

Для реализации принципа минимальных привилегий создадим две роли, соответствующие классам потребителей из ЛР1: «Пользователи» (клиенты) и «Технические специалисты».

Сначала создадим групповые роли без права на вход в систему.

```
CREATE ROLE client_role NOLOGIN;  
  
CREATE ROLE technician_role NOLOGIN;
```

Настройка привилегий:

Доступ к схеме `public` был ограничен для всех пользователей по умолчанию (`REVOKE ALL`). Права выдавались точно на уровне представлений (Views), что позволяет скрыть физическую структуру таблиц от конечных пользователей.

Запрещаем всем по умолчанию доступ к объектам схемы `public`

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
```

Разрешаем ролям использовать схему

```
GRANT USAGE ON SCHEMA public TO client_role;  
GRANT USAGE ON SCHEMA public TO technician_role;
```

Выдаем права на представления для клиентов

```
GRANT SELECT ON public.user_trip_history TO client_role;  
GRANT SELECT ON public.available_scooters TO client_role;
```

Выдаем права на представления для тех. специалистов

```
GRANT SELECT ON public.scooters_for_charging TO technician_role;  
GRANT SELECT ON public.maintenance_log TO technician_role;
```

Наконец, создадим конкретных пользователей с правом входа и добавим их в соответствующие группы.

```
CREATE ROLE anna_vasilieva_client WITH LOGIN PASSWORD 'clientpass123';  
GRANT client_role TO anna_vasilieva_client;  
  
CREATE ROLE sergey_mikhailov_tech WITH LOGIN PASSWORD 'techpass456';  
GRANT technician_role TO sergey_mikhailov_tech;
```

Проверим работу настроенных ограничений, подключаясь от имени созданных пользователей.

```
scooter_rental_db=# SET ROLE anna_vasilieva_client;
SET
scooter_rental_db=> SELECT * FROM public.available_scooters LIMIT 2;
Текущая кодовая страница: 1251
 scooterid | Местоположение | Уровень заряда | Тариф | Стоимость минуты
-----+-----+-----+-----+-----
          2 | Центральный район |          100 | Стандарт |          5.00
          3 | Адмиралтейский район |          45 | Стандарт |          5.00
(2 строки)

scooter_rental_db=> SELECT * FROM public."User" LIMIT 2;
ОШИБКА: нет доступа к таблице User
scooter_rental_db=> SELECT * FROM public.maintenance_log LIMIT 2;
ОШИБКА: нет доступа к представлению maintenance_log
```

Рисунок 9 – Проверка роли anna_vasilieva_client

```
scooter_rental_db=> SET ROLE sergey_mikhailov_tech;
SET
scooter_rental_db=> SELECT * FROM public.scooters_for_charging LIMIT 2;
Текущая кодовая страница: 1251
 scooterid | model | Текущий уровень заряда | Название зоны
-----+-----+-----+-----
(0 строк)

scooter_rental_db=> SELECT * FROM public.Trip LIMIT 2;
ОШИБКА: нет доступа к таблице trip
scooter_rental_db=> SELECT * FROM public.action_log LIMIT 2;
ОШИБКА: нет доступа к таблице action_log
```

Рисунок 10 – Проверка роли sergey_mikhailov_tech

Демонстрация подтверждает, что реализованная схема разграничения доступа эффективно изолирует информационные потоки различных групп пользователей, повышая общую защищенность системы.

4 РЕАЛИЗАЦИЯ СЕРВИСА ДЛЯ ВЗАИМОДЕЙСТВИЯ С РАЗРАБОТАННОЙ БАЗОЙ ДАННЫХ

В данном разделе описывается разработка программного комплекса (API-сервиса и веб-интерфейса), обеспечивающего безопасное взаимодействие конечных пользователей с базой данных.

4.1 Обоснование выбора стека технологий

Архитектура программного комплекса построена на основе микросервисного подхода с использованием следующего стека технологий:

- **Язык программирования: Python.** Выбор обусловлен его простотой, большим количеством библиотек для веб-разработки и работы с данными, а также активным сообществом. Python позволяет быстро прототипировать и создавать надежные серверные приложения.
- **Веб-фреймворк: Flask.** Являясь микро-фреймворком, Flask предоставляет легковесность и гибкость. Он идеально подходит для создания REST API, не навязывая строгую структуру проекта и позволяя разработчику полностью контролировать компоненты приложения.
- **ORM-система: SQLAlchemy.** Это мощная и гибкая ORM-библиотека для Python, позволяющая работать с реляционными базами данных, используя объектно-ориентированный подход. Использование SQLAlchemy абстрагирует разработчика от написания "сырых" SQL-запросов, что является ключевой мерой защиты от атак класса SQL-инъекций.
- **Драйвер для PostgreSQL: psycopg2-binary.** Является наиболее популярным и стабильным драйвером для подключения Python-приложений к СУБД PostgreSQL, обеспечивая надежное и производительное соединение.
- **Frontend: HTML, CSS, JavaScript (vanilla).** Для создания пользовательского интерфейса не использовались тяжеловесные frontend-фреймворки. Применение нативного JavaScript позволяет продемонстрировать логику взаимодействия с API (отправка запросов, обработка JWT, рендеринг данных).

Данный стек технологий является современным, хорошо документированным и позволяет эффективно решать поставленные задачи, уделяя особое внимание аспектам информационной безопасности.

Для отслеживания всех изменений в данных (операций INSERT, UPDATE, DELETE) будет создана система аудита на основе триггеров.

4.2 Архитектура сервиса и описание API

Взаимодействие с базой данных `scooter_rental_db` осуществляется через SQLAlchemy. Первым шагом является маппинг — описание структуры таблиц БД в виде Python-классов. Это позволяет работать с записями в базе данных как с объектами Python, что упрощает разработку и повышает безопасность.

ORM-модели:

Каждая таблица БД имеет соответствующий класс-модель. Ниже приведен фрагмент файла `models.py`, описывающий сущность пользователя:

```
class User(Base):
    __tablename__ = 'User'
    userid = Column('UserID', Integer, primary_key=True)
    fio = Column('FIO', String(255), nullable=False)
    phonenumber = Column('PhoneNumber', String(20), unique=True,
        nullable=False)
    email = Column('Email', String(100), unique=True, nullable=False)
    registrationdate = Column('RegistrationDate', DateTime, nullable=False,
        server_default=func.now())
    lastlogin = Column('LastLogin', DateTime)
```

(Полный листинг моделей приведен в Приложении)

Управление сессиями и безопасное подключение:

Ключевой особенностью реализации является динамическое переключение ролей СУБД. Функция `get_session` иницирует подключение не от имени администратора, а от имени конкретной роли (`client_role` или `technician_role`), полученной после аутентификации.

```
def get_session(role: str, password: str):
    """
    Создает и возвращает сессию SQLAlchemy для указанной роли PostgreSQL.
    Это ключевой механизм безопасности, использующий ролевую модель СУБД.

    :param role: Имя роли (пользователя) в PostgreSQL.
    :param password: Пароль для этой роли.
    :return: Объект сессии SQLAlchemy или None в случае ошибки подключения.
    """
```

```

db_url =
    f"postgresql+psycopg2://{role}:{password}@localhost:7777/scooter_rental_
    db"
try:
    engine = create_engine(db_url)

    connection = engine.connect()
    connection.close()

    SessionLocal = sessionmaker(autocommit=False, autoflush=False,
    bind=engine)
    return SessionLocal()
except OperationalError as e:
    print(f"ОШИБКА: Не удалось подключиться к базе данных для пользователя
    '{role}': {e}")
    return None

```

Работа сервиса демонстрирует, что механизмы защиты, реализованные на уровне СУБД, функционируют корректно при взаимодействии через API.

Пользователь вводит свои учетные данные в веб-интерфейсе. JavaScript отправляет их на эндпоинт `/login`. Сервер проверяет данные и в случае успеха возвращает JWT-токен, который сохраняется в браузере.

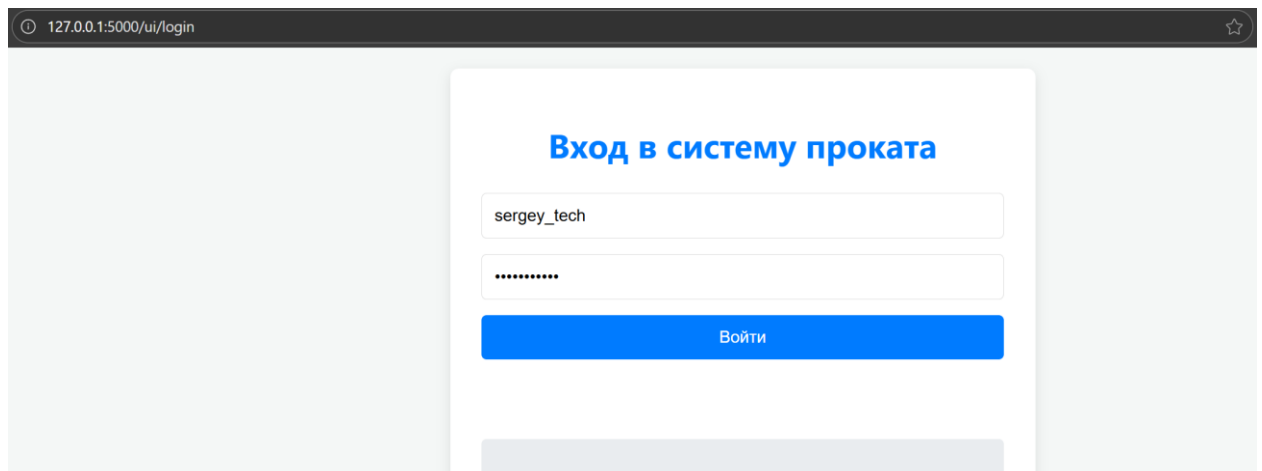
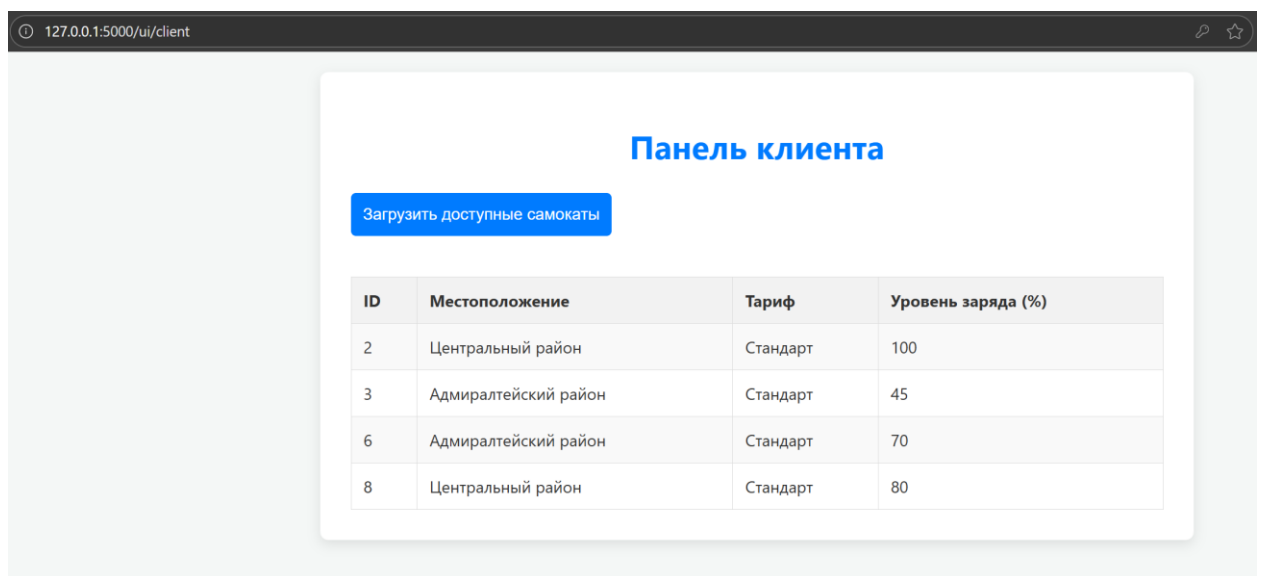


Рисунок 11 – Успешный вход в систему на странице `login.html`

– После входа пользователь-клиент перенаправляется на свою панель. При нажатии на кнопку "Загрузить доступные самокаты" JavaScript отправляет GET-запрос на эндпоинт `/api/client/scooters`, прикрепив JWT-токен в заголовок `Authorization`. Бэкенд:

- Проверяет валидность токена.
 - Извлекает из токена роль "client".
 - Подключается к СУБД от имени пользователя с ограниченными правами (anna_vasilieva_client).
 - Выполняет запрос к представлению available_scooters, на которое у данной роли есть права SELECT.
 - Возвращает данные, которые отображаются в виде таблицы.
- Это подтверждает корректную работу системы разграничения доступа.



ID	Местоположение	Тариф	Уровень заряда (%)
2	Центральный район	Стандарт	100
3	Адмиралтейский район	Стандарт	45
6	Адмиралтейский район	Стандарт	70
8	Центральный район	Стандарт	80

Рисунок 12 – Панель клиента (client_dashboard.html) после успешной загрузки данных

Хотя в интерфейсе не реализованы функции изменения данных, корректная работа триггеров логирования была проверена на этапе разработки. Любая операция INSERT, UPDATE или DELETE, выполненная через сессию SQLAlchemy, корректно фиксируется в таблице action_log. Например, при добавлении новой записи о техобслуживании в лог записывается информация о типе операции ('I'), времени, пользователя (db_user), имени таблицы (maintenance) и JSON-представление новых данных.

5 РЕЗЕРВИРОВАНИЕ БАЗЫ ДАННЫХ И ВОССТАНОВЛЕНИЕ ПО КОНТРОЛЬНЫМ ТОЧКАМ

Обеспечение доступности и целостности данных является одним из ключевых аспектов информационной безопасности. В данном разделе рассматривается реализация стратегии резервного копирования (Backup) и восстановления (Recovery) для разработанной системы.

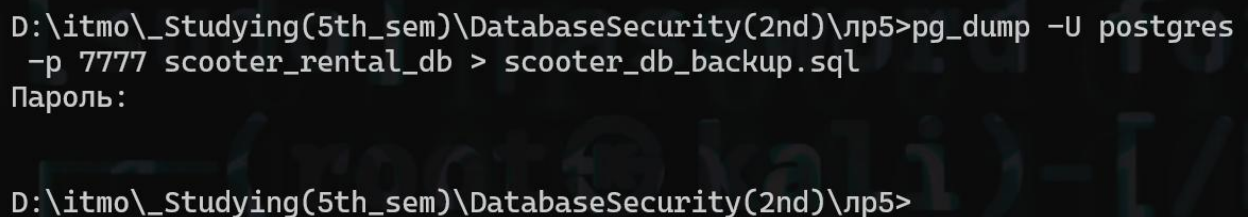
5.1 Создание резервной копии (контрольной точки)

Для создания резервной копии базы данных `scooter_rental_db` была использована стандартная утилита PostgreSQL `pg_dump`. Данная утилита создает логический бэкап — текстовый файл, содержащий SQL-команды, необходимые для полного воссоздания структуры и данных базы на момент копирования. Этот подход является универсальным и позволяет восстанавливать базу данных на различных версиях PostgreSQL и даже на других архитектурах.

Процедура создания бэкапа:

В командной строке была выполнена следующая команда для создания файла резервной копии `scooter_db_backup.sql`, который является контрольной точкой восстановления:

```
pg_dump -U postgres -p 7777 scooter_rental_db > scooter_db_backup.sql
```



```
D:\itmo\_Studying(5th_sem)\DatabaseSecurity(2nd)\лр5>pg_dump -U postgres
-p 7777 scooter_rental_db > scooter_db_backup.sql
Пароль:
D:\itmo\_Studying(5th_sem)\DatabaseSecurity(2nd)\лр5>
```

Рисунок 13 – Создание резервной копии базы данных

5.2 Демонстрация процесса восстановления

После создания контрольной точки в базу данных были внесены преднамеренные деструктивные изменения для имитации случайной ошибки или вредоносных действий.

Симуляция инцидента:

- Удален пользователь с `UserID = 1`.
- Статус самоката с `ScooterID = 2` изменен на `'BROKEN_FOREVER'`.

Query

Query History

```
1 -- 1. "Испортим" данные пользователя
2 UPDATE public."User"
3 SET
4     "fio" = 'DELETED USER',
5     "email" = 'deleted@deleted.com'
6 WHERE userid = 1;
7
8 -- 2. Изменим статус рабочего самоката на бессмысленный
9 UPDATE public.scooter
10 SET
11     status = 'BROKEN_FOREVER'
12 WHERE scooterid = 2;
13
14 -- 3. Проверим изменения
15 SELECT * FROM public."User" WHERE userid = 1; -- Увидишь "испорченные" данные
16 SELECT scooterid, status FROM public.scooter WHERE scooterid = 2; -- Увидишь 'BROKEN_FOREVER'
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	scooterid [PK] integer	status character varying (20)
1	2	BROKEN_FOREVER

Рисунок 14 – База данных после внесения деструктивных изменений

Процедура восстановления:

Для отката изменений была выполнена процедура полного восстановления из созданной ранее резервной копии. Процедура состояла из трех шагов:

- Удаление поврежденной базы данных (`dropdb`).
- Создание новой пустой базы данных с тем же именем (`createdb`).
- Загрузка данных из файла резервной копии в новую базу (`psql`).

Query

Query History

1

-- 3. Проверим изменения

2

SELECT * FROM public."User" WHERE userid = 1;

3

SELECT scooterid, status FROM public.scooter WHERE scooterid = 2;

Data Output

Messages

Notifications

≡+

SQL

	scooterid [PK] integer	status character varying (20)
1	2	available

Рисунок 15 – База данных после успешного восстановления

После выполнения процедуры восстановления из резервной копии был проведен повторный запрос к таблице

Анализ влияния восстановления на журнал аудита:

После восстановления системы была проведена проверка содержимого таблицы `action_log`.

The screenshot shows a PostgreSQL query editor with the following SQL query:

```

1 SELECT
2   log_id,
3   operation_type,
4   operation_time,
5   db_user,
6   table_name,
7   new_data
8 FROM
9   public.action_log
10 ORDER BY
11   log_id DESC
12 LIMIT 5;

```

The results are displayed in a table with 7 columns: log_id (PK integer), operation_type (character (1)), operation_time (timestamp without time zone), db_user (text), table_name (text), and new_data (jsonb). The table shows 5 rows of data.

log_id (PK integer)	operation_type (character (1))	operation_time (timestamp without time zone)	db_user (text)	table_name (text)	new_data (jsonb)
5	D	2025-09-29 12:36:59.752963	postgres	maintenance	[null]
4	U	2025-09-29 12:36:05.041405	postgres	scooter	{\"model\": \"Ninebot G30\", \"status\": \"in_use\", \"zoneid\": 1, \"scooterid\": 1, \"chargelevel\": 80}
3	I	2025-09-29 12:34:10.378005	postgres	User	{\"fio\": \"Тестов Тест Тестович\", \"email\": \"test@test.com\", \"userid\": 18, \"lastlogin\": null, \"phonenumber\": \"+79001234567\", \"registrationdate\": \"2025-09-29 12:33:22.248565\"}
2	D	2025-09-29 12:33:22.248565	postgres	User	[null]
1	I	2025-09-29 12:16:09.172609	postgres	User	{\"fio\": \"Тест Тест Тестович\", \"email\": \"test@test.com\", \"userid\": 17, \"lastlogin\": null, \"phonenumber\": \"+79001234567\", \"registrationdate\": \"2025-09-29 12:16:09.172609\"}

Рисунок 16 – Состояние таблицы логов после восстановления

Процедура восстановления базы данных из полной логической резервной копии (pg_dump) полностью заменяет текущее состояние базы, включая все служебные таблицы. Как следствие, таблица action_log, содержащая записи о нежелательных изменениях, была также "откачена" к своему состоянию на момент создания бэкапа.

Это демонстрирует критическую уязвимость внутренней системы логирования: в случае инцидента и последующего восстановления из бэкапа все следы действий, которые привели к инциденту, будут уничтожены вместе с откатываемыми изменениями. Такой лог не может служить надежным источником для расследования инцидентов безопасности.

Использование pg_dump обеспечивает надежное восстановление данных, однако хранение журналов аудита внутри той же базы данных создает критическую уязвимость. В случае восстановления из бэкапа теряется "цифровой след" инцидента.

Для устранения данного недостатка в промышленной эксплуатации рекомендуются следующие меры, выявленные в ходе анализа:

- Point-in-Time Recovery (PITR): Использование более продвинутого механизма резервирования PostgreSQL, основанного на непрерывном архивировании журнала предзаписи (WAL). Это позволяет восстановить базу на любой момент времени, а не только на момент создания бэкапа, и сохраняет полную историю транзакций.
- Внешние системы логирования: В промышленных системах логи аудита не хранятся в той же базе данных, которую они отслеживают. Они в реальном времени отправляются на внешний, защищенный сервер (например, в стек ELK, Splunk, Graylog).

Это гарантирует сохранность логов даже в случае полной компрометации или уничтожения основной базы данных.

6 АУДИТ БЕЗОПАСНОСТИ РЕАЛИЗОВАННОГО РЕШЕНИЯ

В данном разделе проводится комплексный анализ эффективности внедренных мер защиты информации, оцениваются архитектурные преимущества и недостатки разработанной системы, а также формулируются рекомендации по её доработке для внедрения в промышленную эксплуатацию.

6.1 Анализ реализованных методов защиты

В ходе выполнения курсовой работы была построена система эшелонированной защиты (Defense in Depth), охватывающая как уровень хранения данных, так и уровень приложения.

1. Уровень базы данных (PostgreSQL):

На этом уровне защита обеспечивается средствами самой СУБД, что гарантирует безопасность данных даже в случае обхода логики приложения:

- **Ролевая модель доступа (RBAC):** Реализован принцип минимальных привилегий. Созданы изолированные роли (`client_role`, `technician_role`), права которых ограничены исключительно необходимыми операциями чтения через представления (`Views`). Прямой доступ к физическим таблицам закрыт.
- **Аудит событий:** Внедрена система триггеров, фиксирующая все операции модификации данных (`INSERT`, `UPDATE`, `DELETE`). Это обеспечивает неотказуемость действий и сохраняет историю изменений (версионность данных).
- **Криптографическая защита:** Для хранения критически важной информации (API-ключи) применено симметричное шифрование (AES) с использованием модуля `pgcrypto`. Ключи шифрования не хранятся в БД.

2. Уровень приложения (API-сервис):

На уровне взаимодействия с пользователем реализованы следующие механизмы:

- **Защита от SQL-инъекций:** Использование ORM (`SQLAlchemy`) обеспечивает автоматическую параметризацию запросов, что исключает внедрение вредоносного кода.
- **Аутентификация:** Внедрен механизм JWT (JSON Web Token), позволяющий безопасно передавать контекст пользователя без постоянной пересылки учетных данных.
- **Динамическое управление соединениями:** Сервис не использует единую учетную запись администратора. Подключение к БД осуществляется от имени конкретной роли, соответствующей правам авторизованного пользователя.

6.2 Преимущества и недостатки выбранных решений

На основе проведенного тестирования и имитации инцидентов (раздел 5) были выявлены сильные и слабые стороны архитектуры.

Преимущества:

- **Устойчивость к компрометации приложения:** Даже если злоумышленник получит контроль над сервером приложений и перехватит соединение с БД, он будет ограничен правами текущей роли (например, не сможет удалить таблицы или прочесть чужие данные, если подключен как клиент).
- **Независимость аудита:** Логирование выполняется на уровне ядра СУБД. Это означает, что администратор приложения или разработчик не могут незаметно изменить данные, минуя журнал аудита.
- **Производительность:** Использование нативных средств СУБД (индексы, представления, хранимые процедуры шифрования) работает быстрее, чем реализация аналогичной логики на уровне кода Python.

Недостатки:

- **Уязвимость журналов аудита:** Как показал эксперимент в разделе 5, хранение таблицы логов (`action_log`) внутри той же базы данных является критическим риском. Восстановление из резервной копии (Backup) приводит к потере записей о событиях, произошедших в период между созданием копии и сбоем.
- **Передача данных по открытым каналам:** В текущей реализации (HTTP) трафик не шифруется, что создает риск перехвата токенов авторизации (Man-in-the-Middle attack).
- **Проблема отзыва токенов:** Механизм JWT stateless (без сохранения состояния). Это означает, что выданный токен остается валидным до истечения срока жизни, даже если учетная запись пользователя была заблокирована администратором.

6.3 Рекомендации по дальнейшему усилению защиты

Для устранения выявленных уязвимостей и перевода системы в стадию промышленной эксплуатации необходимо реализовать следующие меры:

1. **Внедрение TLS/SSL:** Настройка шифрования транспортного уровня является обязательной для защиты канала связи между клиентом и сервером, а также между сервером приложений и базой данных.

2. **Вынос системы логирования:** Необходимо настроить репликацию логов в режиме реального времени во внешнюю систему (SIEM-система или выделенный Log-сервер). Это обеспечит сохранность доказательной базы при любых манипуляциях с основной БД.

3. **Реализация списка отзыва (Token Blacklist):** Для решения проблемы блокировки пользователей следует внедрить быстрое хранилище (например, Redis) для хранения идентификаторов отозванных токенов с проверкой при каждом запросе.

4. **Использование PITR (Point-in-Time Recovery):** Переход от простых логических бэкапов к непрерывному архивированию WAL-файлов PostgreSQL, что позволит восстанавливать состояние системы на любой момент времени с точностью до транзакции.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы была решена задача проектирования и реализации защищенной базы данных для информационной системы «Прокат электросамокатов».

В ходе работы были достигнуты следующие результаты:

1. Спроектирована инфологическая модель предметной области, схема данных нормализована до 3НФ.
2. Выполнена физическая реализация базы данных в СУБД PostgreSQL, наполнение тестовыми данными и настройка индексов.
3. Внедрен комплекс мер защиты: мониторинг операций, шифрование критичных данных и разграничение доступа на уровне ролей СУБД.
4. Разработан API-сервис и веб-интерфейс, демонстрирующие безопасное взаимодействие с базой данных с использованием JWT-аутентификации.
5. Протестированы сценарии резервного копирования и восстановления, выявлены особенности влияния восстановления на целостность журналов аудита.

Созданная система демонстрирует устойчивость к основным векторам атак (SQL-инъекции, несанкционированный доступ) и обеспечивает целостность и доступность данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с.
2. Базы данных: Учебник для высших учебных заведений / Под ред. проф. А. Д. Хомоненко. — 6-е изд., доп. - СПб.: КОРОНА-Век, 2009. — 736 с.
3. Psycopg – PostgreSQL database adapter for Python : официальная документация [Электронный ресурс] / The Psycopg Team. – URL: <https://www.psycopg.org/docs/> (дата обращения: 28.09.2025)
4. PostgreSQL 16 Documentation. F.28. pgcrypto. [Электронный ресурс] – URL: <https://www.postgresql.org/docs/current/pgcrypto.html> (дата обращения: 29.09.2025).
5. Официальная документация Flask [Электронный ресурс] – URL: <https://flask.palletsprojects.com/> (дата обращения: 10.10.2025).
6. Официальная документация SQLAlchemy [Электронный ресурс] – URL: <https://www.sqlalchemy.org/> (дата обращения: 10.10.2025).

ПРИЛОЖЕНИЕ А

Листинги основного программного кода

Листинг А.1 – test_db.py

```
import pytest
import psycopg2

# --- Параметры подключения к БД ---
DB_PARAMS = {
    "host": "localhost",
    "port": 5252,
    "user": "postgres",
    "password": "пароль",
    "dbname": "scooter_rental_db"
}

@pytest.fixture(scope="module")
def db_connection():
    """Фикстура для создания и закрытия соединения с БД."""
    try:
        conn = psycopg2.connect(**DB_PARAMS)
        yield conn
        conn.close()
    except psycopg2.OperationalError as e:
        pytest.fail(f"Не удалось подключиться к базе данных: {e}")

def test_connection_is_successful(db_connection):
    """Проверяет, что соединение с БД успешно установлено."""
    assert db_connection is not None and not db_connection.closed

def test_users_table_is_populated(db_connection):
    """Проверяет, что таблица "User" содержит данные (не пустая)."""
    with db_connection.cursor() as cur:
        cur.execute('SELECT COUNT(*) FROM public."User"')
        user_count = cur.fetchone()[0]
        assert user_count >= 8, "В таблице User должно быть не менее 8 записей"

def test_scooters_for_charging_view_logic(db_connection):
```

```

        """Проверяет логику представления 'scooters_for_charging': все самокаты в
        нем должны иметь заряд < 30%."""
        with db_connection.cursor() as cur:
            cur.execute('SELECT "Текущий уровень заряда" FROM
            public.scooters_for_charging')
            low_charge_levels = cur.fetchall()
            for level in low_charge_levels:
                assert level[0] < 30, f"Найден самокат с уровнем заряда
                {level[0]}, что не соответствует логике представления"

def test_foreign_key_constraint(db_connection):
    """Проверяет работу ограничения внешнего ключа, пытаюсь вставить поездку
    с несуществующим UserID."""
    with pytest.raises(psycopg2.errors.ForeignKeyViolation):
        with db_connection.cursor() as cur:
            invalid_user_id = 999999
            cur.execute(
                "INSERT INTO public.Trip (UserID, ScooterID, TariffID,
                StartTime) VALUES (%s, 1, 1, NOW())",
                (invalid_user_id,)
            )
        db_connection.rollback()

def test_unique_email_constraint(db_connection):
    """Проверяет работу ограничения UNIQUE для Email в таблице "User",
    пытаюсь вставить дубликат."""
    with pytest.raises(psycopg2.errors.UniqueViolation):
        with db_connection.cursor() as cur:
            existing_email = 'ivanov@email.com'
            cur.execute(
                'INSERT INTO public."User" (FIO, PhoneNumber, Email) VALUES
                (%s, %s, %s)',
                ('Тестовый Пользователь', '+700000000000', existing_email)
            )
        db_connection.rollback()

def test_scooter_status_default_value(db_connection):
    """Проверяет, что для нового самоката по умолчанию устанавливается статус
    'available'."""
    with db_connection.cursor() as cur:
        cur.execute(

```

```

        "INSERT INTO public.Scooter (Model, ChargeLevel, ZoneID) VALUES
(%s, %s, %s) RETURNING ScooterID",
        ('Test Model', 100, 1)
    )
    new_scooter_id = cur.fetchone()[0]

    cur.execute("SELECT Status FROM public.Scooter WHERE ScooterID = %s",
(new_scooter_id,))
    status = cur.fetchone()[0]

    assert status == 'available'

    db_connection.rollback()

```

Листинг A.2 – models.py

```

from sqlalchemy import (Column, Integer, String, Text, DateTime, Numeric,
                        ForeignKey, CHAR, create_engine, func, LargeBinary)
from sqlalchemy.orm import declarative_base, sessionmaker, relationship
from sqlalchemy.exc import OperationalError

Base = declarative_base()

# --- Описание ORM-моделей для таблиц БД ---

class Zone(Base):
    __tablename__ = 'zone'
    zoneid = Column('ZoneID', Integer, primary_key=True)
    zonename = Column('ZoneName', String(100), nullable=False)
    boundaries = Column('Boundaries', Text)

class Tariff(Base):
    __tablename__ = 'tariff'
    tariffid = Column('TariffID', Integer, primary_key=True)
    tariffname = Column('TariffName', String(50), nullable=False)
    priceperminute = Column('PricePerMinute', Numeric(5, 2), nullable=False)
    startprice = Column('StartPrice', Numeric(5, 2), nullable=False,
server_default='0.00')

class User(Base):
    __tablename__ = '"User"'
    userid = Column('UserID', Integer, primary_key=True)
    fio = Column('FIO', String(255), nullable=False)
    phonenumber = Column('PhoneNumber', String(20), unique=True,
nullable=False)
    email = Column('Email', String(100), unique=True, nullable=False)
    registrationdate = Column('RegistrationDate', DateTime, nullable=False,
server_default=func.now())
    lastlogin = Column('LastLogin', DateTime)

class Scooter(Base):
    __tablename__ = 'scooter'
    scooterid = Column('ScooterID', Integer, primary_key=True)
    model = Column('Model', String(50), nullable=False)

```

```

        chargelevel = Column('ChargeLevel', Integer, nullable=False)
        status = Column('Status', String(20), nullable=False,
server_default='available')
        zoneid = Column('ZoneID', Integer, ForeignKey('zone.ZoneID'))
        zone = relationship("Zone")

class Maintenance(Base):
    __tablename__ = 'maintenance'
    maintenanceid = Column('MaintenanceID', Integer, primary_key=True)
    scooterid = Column('ScooterID', Integer, ForeignKey('scooter.ScooterID'),
nullable=False)
    maintenancetype = Column('MaintenanceType', String(100), nullable=False)
    maintenancedate = Column('MaintenanceDate', DateTime, nullable=False)
    description = Column('Description', Text)
    scooter = relationship("Scooter")

class Trip(Base):
    __tablename__ = 'trip'
    tripid = Column('TripID', Integer, primary_key=True)
    userid = Column('UserID', Integer, ForeignKey('"User".UserID'),
nullable=False)
    scooterid = Column('ScooterID', Integer, ForeignKey('scooter.ScooterID'),
nullable=False)
    tariffid = Column('TariffID', Integer, ForeignKey('tariff.TariffID'),
nullable=False)
    starttime = Column('StartTime', DateTime, nullable=False)
    endtime = Column('EndTime', DateTime)
    cost = Column('Cost', Numeric(8, 2))
    user = relationship("User")
    scooter = relationship("Scooter")
    tariff = relationship("Tariff")

class Payment(Base):
    __tablename__ = 'payment'
    paymentid = Column('PaymentID', Integer, primary_key=True)
    tripid = Column('TripID', Integer, ForeignKey('trip.TripID'),
nullable=False)
    amount = Column('Amount', Numeric(8, 2), nullable=False)
    paymentdate = Column('PaymentDate', DateTime, nullable=False,
server_default=func.now())
    paymentstatus = Column('PaymentStatus', String(20), nullable=False,
server_default='completed')
    trip = relationship("Trip")

class ActionLog(Base):
    __tablename__ = 'action_log'
    log_id = Column(Integer, primary_key=True)
    operation_type = Column(CHAR(1), nullable=False)
    operation_time = Column(DateTime, nullable=False,
server_default=func.now())
    db_user = Column(Text, nullable=False,
server_default=func.current_user())
    table_name = Column(Text, nullable=False)
    old_data = Column(Text)
    new_data = Column(Text)

class UserApiKey(Base):
    __tablename__ = 'user_api_keys'
    key_id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('"User".UserID'), nullable=False)
    api_key = Column(LargeBinary, nullable=False)
    created_at = Column(DateTime, nullable=False, server_default=func.now())

```

```
# --- Функция для создания сессии подключения к БД ---

def get_session(role: str, password: str):
    """
    Создает и возвращает сессию SQLAlchemy для указанной роли PostgreSQL.
    Это ключевой механизм безопасности, использующий ролевую модель СУБД.

    :param role: Имя роли (пользователя) в PostgreSQL.
    :param password: Пароль для этой роли.
    :return: Объект сессии SQLAlchemy или None в случае ошибки подключения.
    """
    db_url =
f"postgresql+psycopg2://{role}:{password}@localhost:7777/scooter_rental_db"
    try:
        engine = create_engine(db_url)

        connection = engine.connect()
        connection.close()

        SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
        return SessionLocal()
    except OperationalError as e:
        print(f"ОШИБКА: Не удалось подключиться к базе данных для
пользователя '{role}': {e}")
        return None
```

Листинг A.3 – app.py

```
import datetime
import json
import os # Для безопасного получения секретного ключа
from functools import wraps
from decimal import Decimal
import jwt
from flask import Flask, Response, request, render_template
from sqlalchemy import text
from werkzeug.security import generate_password_hash, check_password_hash
from models import get_session

# --- Конфигурация Flask приложения ---
app = Flask(__name__)

app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY',
'my_super_secret_for_jwt_in_lab_4_default')
app.config['JSON_AS_ASCII'] = False

# --- "База данных" пользователей API ---
# ВАЖНО: В реальном приложении эта информация должна храниться в таблице БД.
# Для лабораторной работы это допустимое упрощение.
api_users = {
    "anna_client": {
        "password_hash": generate_password_hash("clientpass123"),
        "role": "client",
        "db_user": "anna_vasilieva_client",
        "db_pass": "clientpass123"
    },
    "sergey_tech": {
```

```

        "password_hash": generate_password_hash("techpass456"),
        "role": "technician",
        "db_user": "sergey_mikhailov_tech",
        "db_pass": "techpass456"
    }
}

# --- Вспомогательные функции и декораторы ---
def create_json_response(data, status_code=200):
    """Централизованно создает Flask Response с корректным заголовком и
    кодировкой UTF-8."""
    json_data = json.dumps(data, ensure_ascii=False, indent=2)
    return Response(json_data, mimetype='application/json; charset=utf-8',
                    status=status_code)

def token_required(f):
    """Декоратор для защиты эндпоинтов, требующих JWT-аутентификации."""
    @wraps(f)
    def decorated(*args, **kwargs):
        token = None
        if 'Authorization' in request.headers and
request.headers['Authorization'].startswith('Bearer '):
            token = request.headers['Authorization'].split(" ")[1]

        if not token:
            return create_json_response({'message': 'Токен аутентификации
отсутствует!'}, 401)

        try:
            data = jwt.decode(token, app.config['SECRET_KEY'],
algorithm="HS256")
            current_user = api_users.get(data['user'])
            if not current_user:
                return create_json_response({'message': 'Пользователь из
токена не найден!'}, 401)
            except jwt.ExpiredSignatureError:
                return create_json_response({'message': 'Срок действия токена
истек!'}, 401)
            except jwt.InvalidTokenError:
                return create_json_response({'message': 'Токен недействителен!'},
401)

            return f(current_user, *args, **kwargs)
        return decorated

# --- Маршруты для UI ---
@app.route('/ui/login')
def ui_login_page():
    return render_template('login.html')

@app.route('/ui/client')
def ui_client_dashboard_page():
    return render_template('client_dashboard.html')

@app.route('/ui/technician')
def ui_technician_dashboard_page():
    return render_template('technician_dashboard.html')

# --- Маршруты для API ---
@app.route('/')
def index_page():

```

```

        """Главная страница API, предоставляет информацию о доступных
        эндпоинтах."""
        data = {
            "message": "API для системы проката самокатов",
            "api_endpoints": {
                "POST /login": "Получение JWT токена (Basic Auth)",
                "GET /api/client/scooters": "Получить доступные самокаты (для
клиентов)",
                "GET /api/tech/maintenance_log": "Получить журнал ТО (для
техников)"
            },
            "ui_entry_point": {
                "login_page": "/ui/login"
            }
        }
        return create_json_response(data)

@app.route('/login', methods=['POST'])
def login():
    """Аутентифицирует пользователя по Basic Auth и возвращает JWT."""
    auth = request.authorization
    if not auth or not auth.username or not auth.password:
        return create_json_response({'message': 'Необходима Basic-
аутентификация'}, 401)

    user_data = api_users.get(auth.username)
    if not user_data or not check_password_hash(user_data['password_hash'],
auth.password):
        return create_json_response({'message': 'Неверное имя пользователя
или пароль'}, 401)

    token = jwt.encode({
        'user': auth.username,
        'role': user_data['role'],
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
    }, app.config['SECRET_KEY'], algorithm="HS256")

    return create_json_response({'token': token})

@app.route('/api/client/scooters', methods=['GET'])
@token_required
def get_available_scooters(current_user):
    """Возвращает список доступных самокатов. Доступно только для роли
'client'."""
    if current_user['role'] != 'client':
        return create_json_response({'message': 'Доступ запрещен: функция
только для клиентов'}, 403)

    session = get_session(current_user['db_user'], current_user['db_pass'])
    if not session:
        return create_json_response({'message': 'Ошибка подключения к БД'},
500)

    try:
        query = text("SELECT * FROM public.available_scooters")
        scooters_data = session.execute(query).fetchall()

        result = []
        for row in scooters_data:
            row_dict = dict(row._mapping)
            for key, value in row_dict.items():
                if isinstance(value, Decimal):

```

```

        row_dict[key] = float(value)
    result.append(row_dict)

    return create_json_response(result)
except Exception as e:
    print(f"Error in get_available_scooters: {e}")
    return create_json_response({'message': 'Внутренняя ошибка сервера
при выполнении запроса'}, 500)
finally:
    if session:
        session.close()

@app.route('/api/tech/maintenance_log', methods=['GET'])
@token_required
def get_maintenance_log(current_user):
    """Возвращает журнал ТО. Доступно только для роли 'technician'."""
    if current_user['role'] != 'technician':
        return create_json_response({'message': 'Доступ запрещен: функция
только для техников'}, 403)

    session = get_session(current_user['db_user'], current_user['db_pass'])
    if not session:
        return create_json_response({'message': 'Ошибка подключения к БД'},
500)

    try:
        query = text("SELECT * FROM public.maintenance_log")
        logs_data = session.execute(query).fetchall()

        result = []
        for row in logs_data:
            row_dict = dict(row._mapping)
            for key, value in row_dict.items():
                if isinstance(value, datetime.date):
                    row_dict[key] = value.isoformat()
            result.append(row_dict)

        return create_json_response(result)
    except Exception as e:
        print(f"Error in get_maintenance_log: {e}")
        return create_json_response({'message': 'Внутренняя ошибка сервера
при выполнении запроса'}, 500)
    finally:
        if session:
            session.close()

```